MIN – Genetic Algorithm #3

Quentin Surdez
ISCL, HEIG-VD
quentin.surdez@heig-vd.ch

Contents

Introduction

Genetic algorithms (GA) are optimization techniques inspired by natural selection that solve complex problems through mimicking natural evolution mechanisms. This report examines GA application in three very different domains: Mastermind, where the GA needs to find a given sentence, Travelling Salesman Problem, where the GA finds a path close to the optimal, and image recreation. I will analyse different configurations and how to translate a problem to a chromosome.

Practical Work

Mastermind GA

Question 1

> *With your code, what would be the chromosome for the sentence "METHINKS IT IS LIKE A WEASEL"?*

For the genetic algorithm to work easily, a mapping between the possible letters and integers is implemented. Each gene can have a value between 0 and 26 as there are 26 uppercase letters possible as well as the space symbol.

Thus, the sentence "METHINKS IT IS LIKE A WEASEL" would have a chromosome like this:

```
[12 4 19 7 8 13 10 18 26 8 19 26 8 18 26 11 8 10 4 26 0 26 22 4 0 18 4 11]
```

Where each integer represent one of the allowed symbols.

Travelling Salesman Problem GA

Question 2.1

> *Provide the better route you found and the shortest path in kilometers. Is it the optimal shortest path ? explain.*

The better route found by the genetic algorithm implemented is the following (it is not the one currently within the notebook as I rerun the algo to test some changes. I kept this one as it shared a lot of similarities to the optimal path found):

```
[ 1  0  9  8 10  7 12  6 11  5  4  3  2 13  1]
```

This route has a length in km of 3354.523137477158, the distance function used is the haversine function. It is used to calculate the distance on a sphere such as the earth.

With just our genetic algorithm, we cannot say if it is or not the optimal shortest path. In deed, we don't have a specific end goal here as the Mastermind had. We only give a problem to our algorithm and hope it will achieve a good result.

As the Travelling Salesman Problem is a NP-Problem, quite well known in optimisation, we could calculate the exact optimal solution from the set of data. The library used to calculate the exact optimal solution is `python_tsp`. With this library we have been able to calculate the optimal path between the 14 cities:

```
[0, 9, 8, 10, 7, 12, 6, 11, 5, 4, 3, 2, 13, 1, 0]
```

This route has a length in km of 3354.5231374771574.

We can see that the two paths are different, however it is only the first city that is different. Otherwise, the rest of the path is shared between the optimal solution and the better path our genetic algorithm found.

This shows that the method using genetic algorithm can have significant good results in very little time. If we were to have 100 cities, and very limited resources, we could argue that the genetic algorithm approach is better than the exact optimal solution.

Question 2.2

> *Describe your fitness function.*

Here's my fitness function:

```
def fitness_function(ga_instance, solution, solution_idx):

      tour_distance = calculate_tour_distance(solution, distance_matrix)

```

```
5       # Return fitness (smaller distance = higher fitness)
6       if tour_distance == 0:
7               return 0
8       return 1.0 / tour_distance
```

We first calculate the distance of the tour given by the solution with the pre-comupted distance matrix. Then, as a smaller distance mean a higher fitness, we divide 1 with the distance found. Thus, the algorithm will try to maximize the fitness value.

Let's see how the calculation of the tour is made:

```
1  def calculate_tour_distance(tour, dist_matrix):
2       distance = 0
3
4       for i in range(NUM_CITIES):
5               from_city = tour[i]
6               to_city = tour[(i + 1) % NUM_CITIES] # the city at index 13 will go
   back to the first city !
7               distance += dist_matrix[from_city][to_city]
8
9       return distance
```

The solution does not have within it the wayback from the last city to the first one. Thus, we iterate on the list to find the from and to city and the to city is modulo the total number of cities. We then have the complete distance with the wayback to the first city.

Question 2.3

> *Explain the way you encoded the solution, give a chromosome example.*

The best way to explain it is by giving the parameters linked to the genes within the constructor of the genetic algorithm:

```
1  gene_space = [list(range(NUM_CITIES))] * NUM_CITIES
2
3  ga_instance = pygad.GA(
4      ...
5      num_genes=NUM_CITIES,
6      gene_space=gene_space,
7      gene_type=int,
8      allow_duplicate_genes=False,
9      ...
10 )
```

We can understand, from this configuration, that the number of genes of one of our chromosome is equal to the number of cities. Then, the range the values can have is from 0 to the number of cities minus 1. And finally they are integers. We do not allow a duplicate within the genes. This guarantees, valid solution as the space is 14 and no genes can have the same values.

In fact, I have already given chromosome example in the precedent answer. In deed, the chromosomes look just like array of numbers from 0 to 13 without any repetition. Here's another one:

```
1  [ 2 13  1  0  9  8 10  7 12  6 11  5  4  3]
```

Question 2.4

> *Provide the configuration of the GA you finally used to find your better results: mutation, crossover, population size, type of selection, mutation, crossover used, number of generations. Describe the methodology or experiments performed in order to get your better results.*

Here's the final configuration of my GA:

```
NUM_GENERATIONS = 1000
POPULATION = 400
PARENTS_MATING = 30
MUTATION_PROBA = 0.2
CROSSOVER_PROBA = 0.9
MUTATION_TYPE = "random"
PARENT_SELECTION_TYPE = "tournament"

gene_space = [list(range(NUM_CITIES))] * NUM_CITIES

ga_instance = pygad.GA(
    sol_per_pop=POPULATION,
    num_genes=NUM_CITIES,
    num_generations=NUM_GENERATIONS,
    num_parents_mating=PARENTS_MATING,
    fitness_func=fitness_function,
    gene_space=gene_space,
    gene_type=int,
    on_generation=on_generation,
    mutation_type=MUTATION_TYPE,
    mutation_probability=MUTATION_PROBA,
    random_mutation_min_val=0,
    random_mutation_max_val=NUM_CITIES,
    crossover_type="single_point",
    crossover_probability=CROSSOVER_PROBA,
    parent_selection_type=PARENT_SELECTION_TYPE,
    K_tournament=5,
    allow_duplicate_genes=False,
    keep_parents=5,
    stop_criteria=["reach_1000", "saturate_50"]
)
```

There are some hyperparameters that I have explored. My methodology was not very strict. I have mainly changed the mutation probability, the crossover probability and the mutation type as well as the parent selection type. The last two parameters are what I have focused on. In deed, the precedent parameters come from both the knowledge gained within the lectures as well as the precedent fine tuning for the mastermind problem.

I have observed that the parent selection type does have an impact on the solution given and its fitness. Some parent selction type such as the steady state selection or the random selection do not attain a score that is as high as the other methods.

Question 2.5

> *Provide relevant plots of your experiments and explanations.*

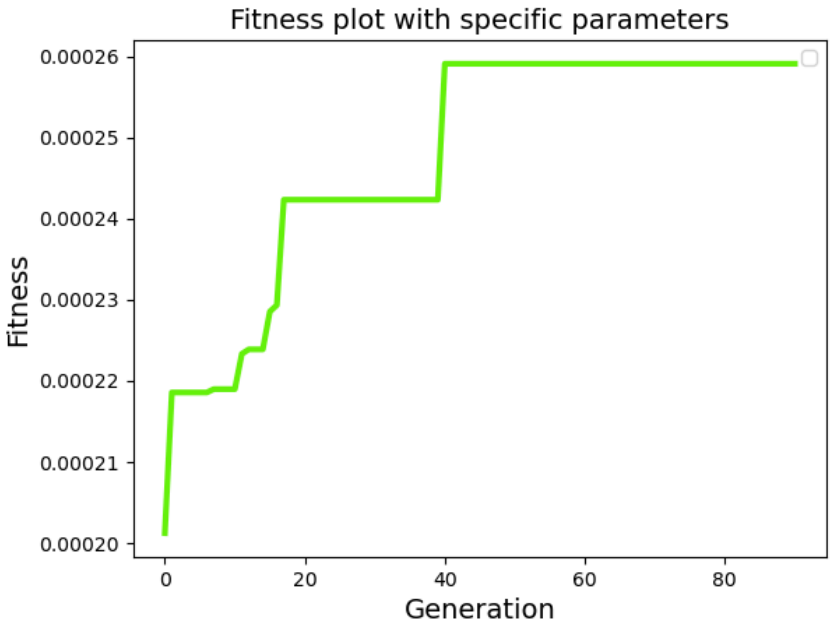

Figure 1: Fitness plot with parent selection type random and mutation type random

We can see in this plot that the fitness of the best solution provided by the GA does not go higher than 0.00026. However, if we observe the behavior of the tournament parent type selection, the fitness can and will go higher.

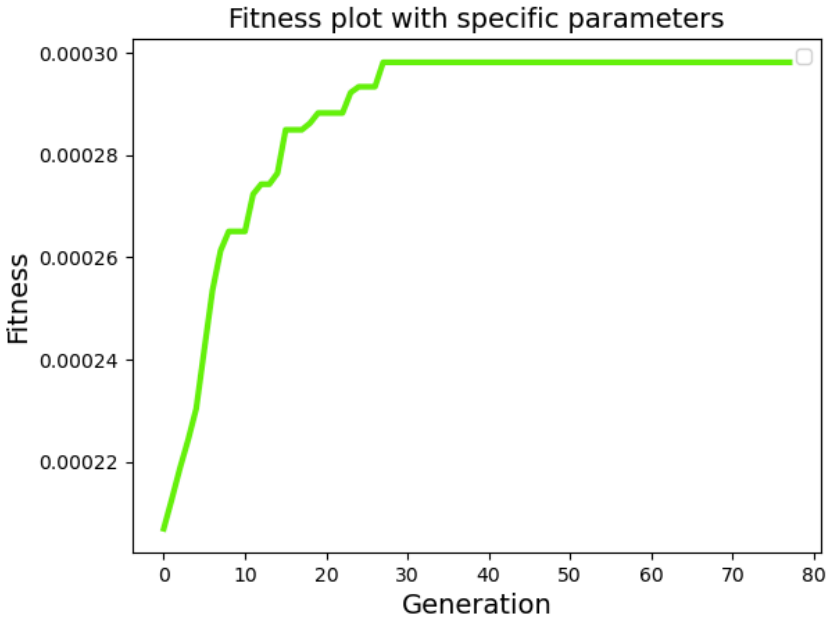

Figure 2: Fitness plot with parent selection type tournament and mutation type random

Here we can observe that the fitness can go as high as 0.00029 which is quite better considering the way we calculate it.

Different mutation type have been explored, but the random or scramble are the ones with the best results. Swap is slightly worse but by a very very little margin.

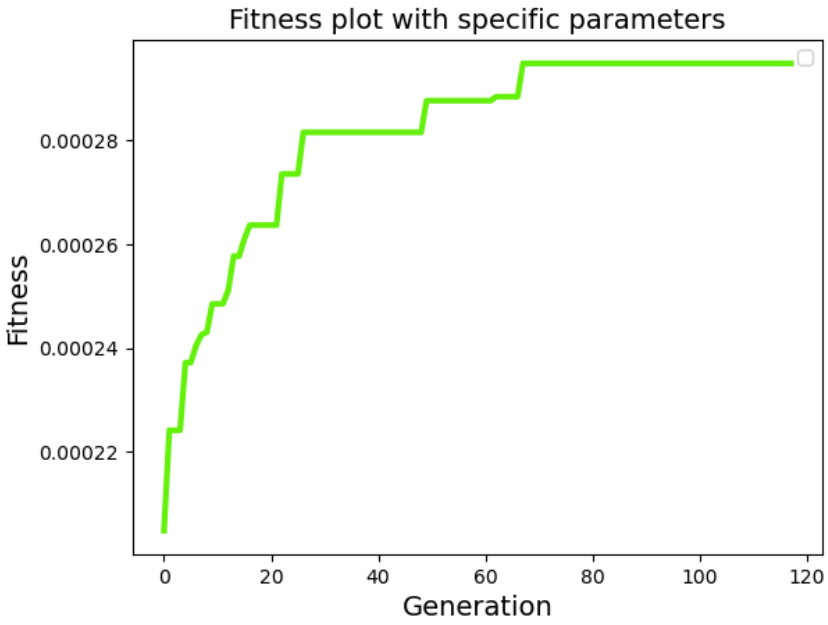

Figure 3: Fitness plot with parent selection type tournament and mutation type swap

These graphs cannot give enough information on wether or not these methods are best for this specific problem. A more thorough exploration of the different combinations of the hyperpareters would be needed to have a good conclusion. However, time is limited and I do what I can with the one provided to me.

MonaLisa GA

Question 3.1

> *The way your extracted the colour palette*

The colour palette, composed of 32 colours, has been extracted by using the KMeans method. The number of clusters is 32.

This method has been used as it is made to classify n observations into k clusters which is exactly what we want to do here. We have n pixels with different color values. Then we want to have all these observations into exactly 32 clusters. Which are our 32 colors.

I have used the `KMeans` class from `scikit-learn`.

We will use this colour palette for the values of our first generation.

Question 3.2

> *Your parameters N_SHAPES_LIST, N_INDIVIDUAL_LIST*

These parameters have been the one that I have changed the most during the testing. I first tried with a very complex and big image (dragon 5Mb). Then I changed with a smaller but still complex image (dragon 125x128). The results were not good with these parameters:

Figure 4: Image used to choose the parameters at first

```
N_SHAPES_LIST = [60, 35, 25, 15, 10, 5]
N_INDIVIDUAL_LIST = [50, 100, 200, 250, 300, 400]
```

I have changed strategy to try with a small and simple image (balloons 190x190). This allowed me to find a specific set of parameters that were working pretty well !

Figure 5: Simpler image used to choose the parameters in a second time

```
N_SHAPES_LIST = [60, 35, 25]
N_INDIVIDUAL_LIST = [75, 150, 300]
```

Question 3.3

> *Your parameters to train the algorithm*

From the test protocol explained in the precedent question, here's the final parameters I have for my genetic algorithm:

```
# RGB + (center_X, center_Y, radius_X, radius_Y)
N_PROPERTIES = 3 + 4 + 1
N_SHAPES_LIST = [60, 35, 25]
N_INDIVIDUAL_LIST = [75, 150, 300]
MUTATION_PROBABILITY = [0.05, 0.01]
CROSSOVER_PROBABILITY = 0.9
PARENT_SELECTION_TYPE = "tournament"
K_TOURNAMENT = 5

ga_instance = pygad.GA(
  ...
  mutation_type="adaptive",
  crossover_type="two_points",
  keep_elitism=5,
  stop_criteria="saturate_50",
  ...
)
```

Question 3.4

> *How you define the chromosomes (what are the genes you defined and what they represent)*

The definition of the genes are taken from the example, I haven't changed much the definition.

The gene space of each chromosome is defined as followed:

```
gene_space = [{'low': 0, 'high': 1}] * N_PROPERTIES * n_shapes
```

For the number of properties and the number of shapes defined, we will have a gene with a value between 0 and 1. Let's look at a concrete example:

The number of properties for our ellipses is 3 genes for the RGB colour, 2 genes for the center x and y coordinates, 2 genes for the x and y radius and finally 1 gene for the angle. These genes code for the colour and the different properties of one ellipse.

Then we have our number of shapes that depend on the step we're at. In our first step we have 60 shapes, thus our chromosome has 60×8 genes all with a lower bound at 0 and an upper bound at 1.

Question 3.5

> *Initial image you choosed and the resulting image you obtained*

Here I will expose the image with the same resolution as the test balloons image as well as an image with the same form but with a higher resolutions. This was done to test if the parameters chosen were applicable for different sizes of images.

The initial image chosen is this image of Spyro the dragon (125x128):

Figure 6: Original image of Spyro the dragon

The image created with the genetic algorithm is the following one:

Figure 7: Generated image with the genetic algorithm

The image of Spyro the dragon with a bigger size (900x900):

Figure 8: Original image of Spyro the dragon (bigger)

The image created with the genetic algorithm is the following one:

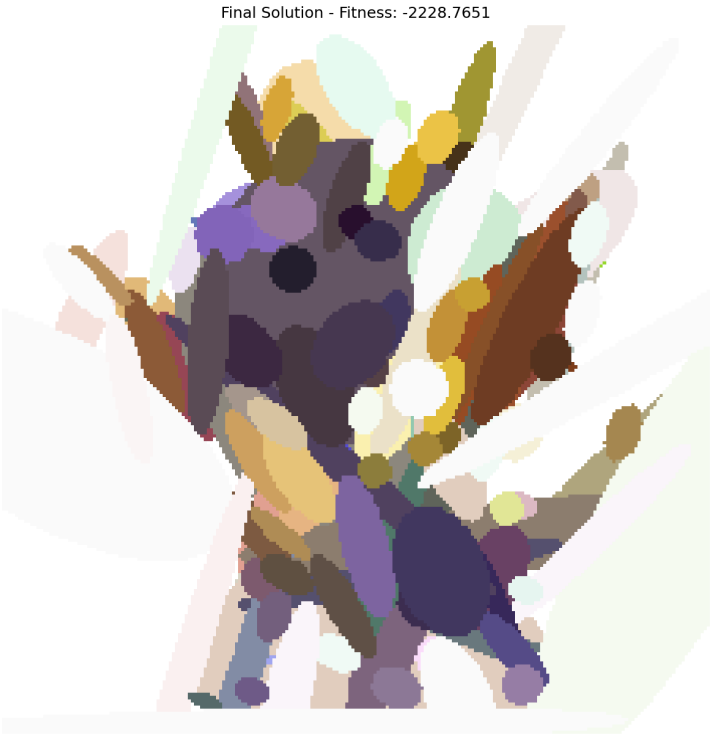

Figure 9: Generated image with the genetic algorithm

Question 3.6

> *Fitness plots*

Here are the fitness plots of each step of the creation of the image in Figure 7.

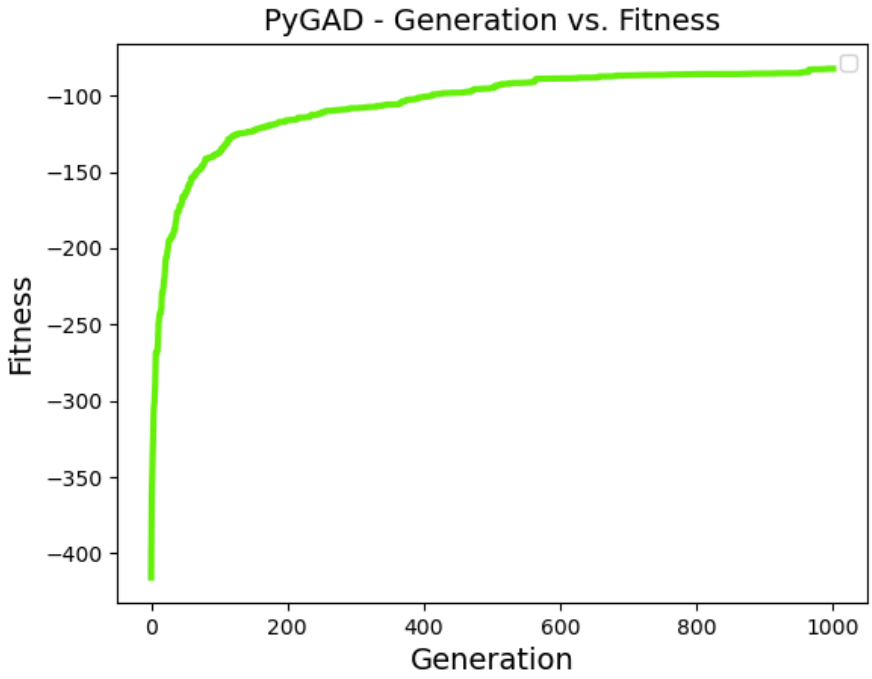

Figure 10: Fitness plot of step 0

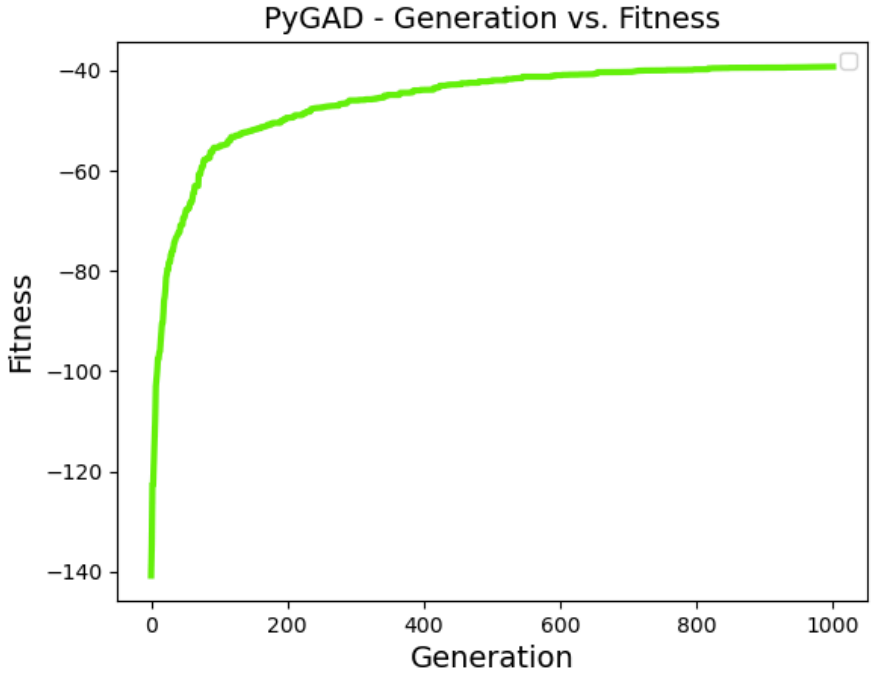

Figure 11: Fitness plot of step 1

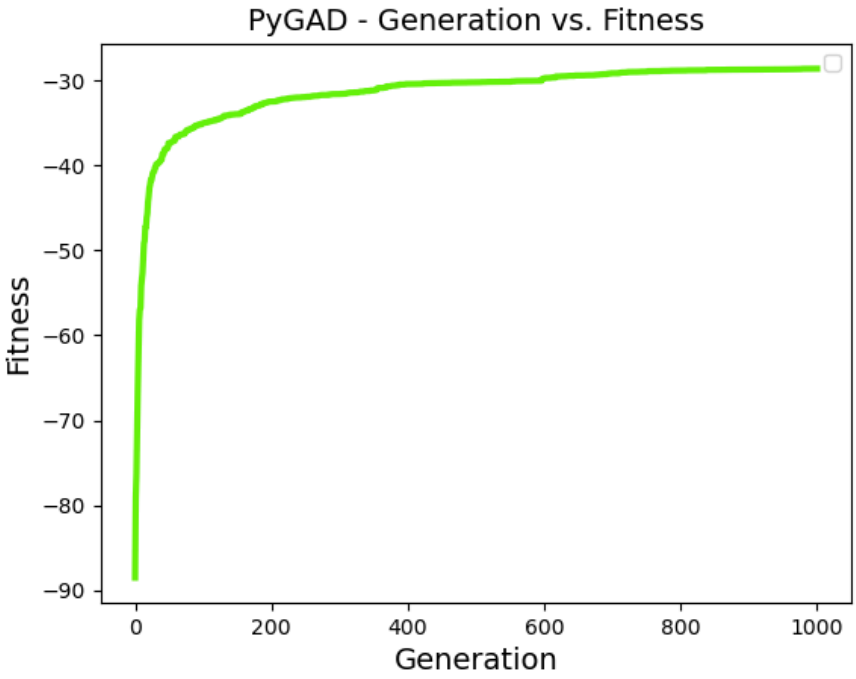

Figure 12: Fitness plot of step 2

We can observe the progression of the fitness throughout the steps with the following figure:

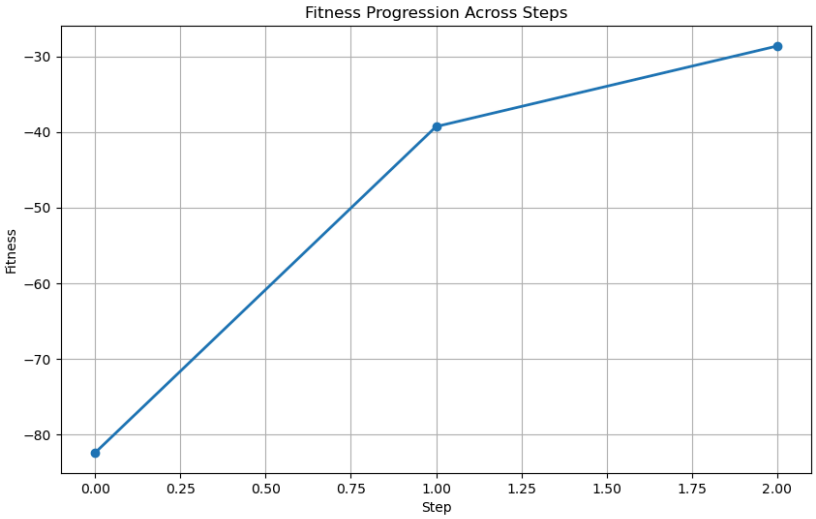

Figure 13: Fitness progression throughout the steps

We can deduce here that the algorithm does converge and that maybe it would benefit from having more steps. However, as my machine has its physical limits, I have not tried to add more steps with even more population as this would take too long.

Conclusion

My GA implementations showed strong results across all problems, with the TSP solution nearly matching the optimal path. Parameter tuning was critical, with parent selection type always being

quite important. These experiments confirm the versatility and effectivness of genetic algorithm for optimization problems.